

Residual-Aware Bayesian Clustering:

Complete Derivations from First Principles

Lorenzo Tomaz
AE Studio

Contents

1	Probability Densities and Likelihood	3
1.1	The Gaussian density	3
1.2	Likelihood: turning a distribution into a score	3
1.3	Conditional distributions	4
2	Latent Variable Models and the Marginal Likelihood	5
2.1	Why latent variables?	5
2.2	Why not just maximize the marginal likelihood?	6
3	The Evidence Lower Bound (ELBO)	7
3.1	The problem: a log of a sum	7
3.2	Introducing a helper distribution	7
3.3	Step-by-step derivation	7
3.4	Decomposing the ELBO	9
4	The EM Algorithm	9
4.1	E-step: optimize over q	9
4.2	M-step: optimize over θ	10
5	Adding Bayesian Priors	11
5.1	Why priors?	11
5.2	The Dirichlet prior on mixing weights	11
5.3	The Normal-Inverse-Wishart (NIW) prior on centers and covariances	12
6	Our Model: Adding the Residual Factor	13
6.1	The generative model	13
6.2	The joint observation likelihood	14
6.3	The log-likelihood of the joint observation	14
6.4	The role of the residual scale	15
7	E-step for Our Model	15
8	M-step for Our Model	16
8.1	Variant A: Taylor-analytic	17
8.2	Variant B: Taylor-LS	17
8.3	Variant C: Local EDMD	19

9	Model Selection	20
9.1	How many clusters?	20
9.2	The NIW marginal likelihood (derivation)	20
10	Summary: What Each Piece Does	20

1 Probability Densities and Likelihood

1.1 The Gaussian density

A d -dimensional Gaussian (normal) distribution with mean $\mu \in \mathbb{R}^d$ and covariance $\Sigma \in \mathbb{R}^{d \times d}$ (symmetric positive definite) has density

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right). \quad (1)$$

- The normalizer $(2\pi)^{d/2} |\Sigma|^{1/2}$ ensures $\int_{\mathbb{R}^d} \mathcal{N}(x; \mu, \Sigma) dx = 1$.
- The exponent $-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)$ is a quadratic form measuring “distance” from μ in the metric defined by Σ^{-1} . Points far from μ (in the Σ -metric) get low density.
- The **precision matrix** Σ^{-1} controls the shape: large eigenvalues of Σ^{-1} (small eigenvalues of Σ) mean the distribution is narrow in that direction.

Intuition 1.1. The Gaussian is the maximum-entropy distribution for a given mean and covariance. If all you know about a random variable is its mean and spread, the Gaussian is the “least-assuming” density consistent with those constraints. This is why it appears everywhere as a default model for continuous noise.

1.2 Likelihood: turning a distribution into a score

Given a fixed observation $x_{\text{obs}} \in \mathbb{R}^d$, we can *evaluate* the density at that point:

$$\ell(\mu, \Sigma) = \mathcal{N}(x_{\text{obs}}; \mu, \Sigma). \quad (2)$$

This number ℓ is called the **likelihood** of the parameters (μ, Σ) given the data x_{obs} . Note:

- The observation x_{obs} is **fixed** (it’s the data we collected).
- The parameters (μ, Σ) are **variable** (we want to find the best ones).
- The likelihood is *not* a probability distribution over (μ, Σ) —it does not integrate to 1 over parameter space.

With P independent observations $x_1, \dots, x_P$, the joint likelihood is the product:

$$\ell(\mu, \Sigma) = \prod_{i=1}^P \mathcal{N}(x_i; \mu, \Sigma). \quad (3)$$

Taking the logarithm (which preserves the location of the maximum):

$$\log \ell(\mu, \Sigma) = \sum_{i=1}^P \log \mathcal{N}(x_i; \mu, \Sigma) = -\frac{Pd}{2} \log 2\pi - \frac{P}{2} \log |\Sigma| - \frac{1}{2} \sum_i (x_i - \mu)^\top \Sigma^{-1} (x_i - \mu). \quad (4)$$

Key idea. Maximum likelihood estimation (MLE): choose (μ, Σ) to maximize $\log \ell$. For a single Gaussian, the MLE is the sample mean $\hat{\mu} = \frac{1}{P} \sum_i x_i$ and sample covariance $\hat{\Sigma} = \frac{1}{P} \sum_i (x_i - \hat{\mu})(x_i - \hat{\mu})^\top$. This is tractable and has a unique closed-form solution.

1.3 Conditional distributions

A **conditional distribution** $p(y | x)$ is a density over y that depends on a given value of x . It satisfies $\int p(y | x) dy = 1$ for each fixed x . The dependence on x enters through the parameters of the density.

Example 1.1 (Linear regression). $y | x \sim \mathcal{N}(w^\top x, \sigma^2)$. The mean $w^\top x$ depends on the input x ; the variance σ^2 does not. For a given x , this is a proper Gaussian density over y .

Example 1.2 (Our residual model). Suppose we have a dynamical system $\dot{x} = f(x)$ and we have partitioned phase space into N clusters. Each cluster k carries a **local model** $L_k: \mathbb{R}^d \rightarrow \mathbb{R}^d$ —a function that, given any point x , outputs a *prediction* of the vector field $f(x)$. The local model is a deterministic function defined by the cluster’s parameters; it is *not* a random variable.

Concretely, L_k is an approximation of f that is built from the cluster’s center c_k and some local dynamical parameters (a Jacobian matrix, an affine operator, or a Koopman operator—the exact form will be specified in Section 6). For now, all that matters is: **given c_k and the local parameters, $L_k(x)$ is a computable function that takes a point $x \in \mathbb{R}^d$ and returns a vector in $\mathbb{R}^d$.** For example, in the simplest case (Taylor linearization):

$$L_k(x) = f(c_k) + \mathbf{J}(c_k)(x - c_k), \quad (5)$$

where $f(c_k) \in \mathbb{R}^d$ is the vector field evaluated at the center (a known vector) and $\mathbf{J}(c_k) \in \mathbb{R}^{d \times d}$ is the Jacobian matrix at the center (a known matrix). The computation is: subtract the center, multiply by the Jacobian, add the vector-field value at the center. The result is an affine function of x .

With L_k defined, we can specify the conditional distribution:

$$f_{\text{obs}}(x_i) | x_i, z_i = k \sim \mathcal{N}(L_k(x_i), \sigma^2 I). \quad (6)$$

This says: given the input x_i and the assignment to cluster k , the observed vector field $f_{\text{obs}}(x_i)$ is a Gaussian random variable with:

- **Center (“mean”)** $= L_k(x_i)$. This is the local model’s prediction of f at x_i . It is computed by evaluating the function L_k at the specific input x_i —**it is a single deterministic value, not an average over anything**. Different inputs give different values.

We call it “the mean” only because of Gaussian convention: in $\mathcal{N}(y; \mu, \Sigma)$, the parameter μ is traditionally called “the mean” because if one drew infinitely many samples $y_1, y_2, \dots$ from this distribution, their sample average would converge to μ . But μ itself is just a location parameter—the point where the bell curve is centered. In our context, a less ambiguous name is **the predicted value**: $L_k(x_i)$ is the value that the local model predicts the vector field should take at x_i , and the Gaussian is centered there.

- **Covariance** $= \sigma^2 I$. This is a fixed isotropic matrix that does *not* depend on x_i . It sets the scale of “tolerable prediction error”—how far the actual observation $f_{\text{obs}}(x_i)$ can be from the prediction $L_k(x_i)$ before the likelihood becomes small.

For a given x_i and k , this is a proper Gaussian density over $f_{\text{obs}} \in \mathbb{R}^d$ (integrates to 1 over $\mathbb{R}^d$).

When we evaluate this density at the *observed* value $f_{\text{obs}}(x_i)$, we get a **likelihood value**:

$$\mathcal{N}(f_{\text{obs}}(x_i); L_k(x_i), \sigma^2 I) = (2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|f_{\text{obs}}(x_i) - L_k(x_i)\|^2}{2\sigma^2}\right). \quad (7)$$

This number depends on three things:

1. $f_{\text{obs}}(x_i)$: the observed vector field (data—fixed).
2. $L_k(x_i)$: the local model’s prediction (computed from current cluster parameters and input x_i).
3. σ^2 : the noise scale (hyperparameter).

It measures “how well does cluster k ’s local model predict the vector field at x_i ?” A small squared error $\|f_{\text{obs}}(x_i) - L_k(x_i)\|^2$ gives a large likelihood; a large squared error gives a small likelihood.

2 Latent Variable Models and the Marginal Likelihood

2.1 Why latent variables?

A **mixture model** assumes that each data point x_i was generated by one of N components, but we don’t observe which one. The identity of the generating component is a **latent (hidden) variable** $z_i \in \{1, \dots, N\}$.

What is x_i ? In our setting, $x_i \in \mathbb{R}^d$ is a **state vector** of the dynamical system—a single point in phase space. For the Lorenz attractor, $x_i = (x, y, z) \in \mathbb{R}^3$ is a 3D position on the attractor. For the damped pendulum, $x_i = (\theta, \dot{\theta}) \in \mathbb{R}^2$ is an angle and angular velocity. We have P such state vectors, sampled from a trajectory or from the phase space directly.

The generative process for a standard Gaussian mixture:

1. Draw $z_i \sim \text{Categorical}(\pi_1, \dots, \pi_N)$, where $\pi_k \geq 0$, $\sum_k \pi_k = 1$.
2. Draw $x_i \sim \mathcal{N}(\mu_{z_i}, \Sigma_{z_i})$.

Step 2 says: given that state x_i was assigned to cluster k (i.e., $z_i = k$), its location in phase space is drawn from a Gaussian centered at μ_k with covariance Σ_k . The term $p(x_i | z_i = k, \theta) = \mathcal{N}(x_i; \mu_k, \Sigma_k)$ answers the question: “*given that this state belongs to cluster k , how likely is it to be at this particular location in phase space?*” States close to the cluster center μ_k get high probability; states far away get low probability.

We observe x_i but not z_i . To write the density of the observed variable x_i alone, we need to **marginalize out** the latent variable z_i .

Definition 2.1 (Marginalization). If two variables A and B have a joint distribution $p(A, B)$, the **marginal distribution** of A is obtained by summing (or integrating) over all possible values of B :

$$p(A) = \sum_{\text{all } b} p(A, B = b). \quad (8)$$

This “removes” B from the joint by accounting for every possibility.

In our case, $A = x_i$ (observed) and $B = z_i$ (latent, takes values $1, \dots, N$). The marginalization formula gives:

$$p(x_i | \theta) = \sum_{k=1}^N p(x_i, z_i = k | \theta). \quad (9)$$

Each term $p(x_i, z_i = k | \theta)$ is the joint probability of observing x_i and having it come from component k . We can decompose this joint using the **chain rule** (also called the product rule) of probability:

$$p(x_i, z_i = k | \theta) = p(z_i = k | \theta) \cdot p(x_i | z_i = k, \theta). \quad (10)$$

This says: the probability of “ x_i was generated by component k ” equals the probability of “component k was chosen” times the probability of “component k generated x_i .” Now we substitute what we know about each factor:

- $p(z_i = k \mid \theta) = \pi_k$ (by the Categorical prior on z_i).
- $p(x_i \mid z_i = k, \theta) = \mathcal{N}(x_i; \mu_k, \Sigma_k)$ (by the Gaussian emission model of component k).

Substituting into (10) and then into (9):

$$p(x_i \mid \theta) = \sum_{k=1}^N \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k). \quad (11)$$

This is the **marginal density** of x_i —a weighted sum of Gaussians, where $\theta = (\pi, \{\mu_k, \Sigma_k\}_{k=1}^N)$ collects all parameters. Each term contributes the density of x_i under component k , weighted by the prior probability π_k of that component.

2.2 Why not just maximize the marginal likelihood?

The log of the marginal likelihood for all data is

$$\log p(X \mid \theta) = \sum_{i=1}^P \log \left(\sum_{k=1}^N \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right). \quad (12)$$

This expression contains a log of a sum, and we claim it has no closed-form maximizer. Let us see concretely why.

For a single Gaussian ($N = 1$), there is no problem. The sum has one term, so

$$\log p(X \mid \theta) = \sum_i \log \mathcal{N}(x_i; \mu, \Sigma).$$

The log applies directly to a single Gaussian—no sum inside. The derivative with respect to μ is:

$$\frac{\partial}{\partial \mu} \sum_i \log \mathcal{N}(x_i; \mu, \Sigma) = \sum_i \Sigma^{-1}(x_i - \mu) = 0 \implies \mu = \frac{1}{P} \sum_i x_i.$$

One equation, one unknown, closed-form solution.

For a mixture ($N > 1$), the parameters are coupled. Now differentiate with respect to μ_k using the chain rule for $\log(\sum_j \dots)$:

$$\frac{\partial}{\partial \mu_k} \log \left(\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j) \right) = \frac{\pi_k \Sigma_k^{-1}(x_i - \mu_k) \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}. \quad (13)$$

The denominator $\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)$ contains **every cluster's parameters**—all μ_j, Σ_j, π_j . Setting the derivative to zero for μ_k gives an equation that involves $\mu_1, \dots, \mu_N, \Sigma_1, \dots, \Sigma_N$, and $\pi_1, \dots, \pi_N$ simultaneously. It is a system of coupled nonlinear equations with no closed-form solution.

Remark 2.1. The ratio $\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k) / \sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)$ is exactly the responsibility r_{ik} . But here it is a *function of all parameters*, not a fixed number. That is the coupling: you cannot solve for μ_k without knowing r_{ik} , and you cannot compute r_{ik} without knowing all the μ_j . EM breaks this circularity by alternating: fix r_{ik} (E-step), then solve for μ_k (M-step), repeat.

Key idea. Latent variable models have intractable marginal likelihoods because the log of a sum couples all cluster parameters together. The EM algorithm breaks this coupling by replacing $\log(\text{sum})$ with $\text{sum}(\log)$ (via the ELBO), at the cost of introducing an auxiliary distribution q that must be iteratively updated.

3 The Evidence Lower Bound (ELBO)

3.1 The problem: a log of a sum

Our goal is to maximize $\log p(X | \theta)$ (12) over θ . But this expression contains a *logarithm of a sum*, which has no closed-form maximizer. We need a workaround.

The strategy: construct a simpler function—a **lower bound** on $\log p(X | \theta)$ —that *is* tractable to maximize. If we can make this bound tight (close to the true value), then maximizing the bound approximately maximizes the true objective.

3.2 Introducing a helper distribution

The key trick is to introduce an auxiliary distribution q over the latent variables. Let $q(z_i)$ be *any* probability distribution over $\{1, \dots, N\}$ —one for each data point i . We write $q(z_i = k)$ for the probability that q assigns point i to cluster k . The only requirements are $q(z_i = k) \geq 0$ and $\sum_k q(z_i = k) = 1$.

Intuition 3.1. Think of q as our current “guess” at the cluster assignments. It does not have to be the true posterior—it can be *anything*. The ELBO derivation works for any q , and we will later choose q to make the bound as tight as possible.

3.3 Step-by-step derivation

We start from the marginal log-likelihood for a single data point and work through five algebraic steps.

Step 1: rewrite using marginalization. The marginal $p(x_i | \theta)$ is a sum over the latent variable (as derived in Section 2.1):

$$\log p(x_i | \theta) = \log \sum_{k=1}^N p(x_i, z_i = k | \theta). \quad (14)$$

This is just the log of the marginal density. So far, nothing new.

Step 2: multiply and divide by $q(z_i = k)$ inside the sum. For any q with $q(z_i = k) > 0$ for all k :

$$\log \sum_k p(x_i, z_i = k | \theta) = \log \sum_k q(z_i = k) \cdot \frac{p(x_i, z_i = k | \theta)}{q(z_i = k)}. \quad (15)$$

This is a purely algebraic identity—multiplying and dividing by the same positive number changes nothing. **Why do we do this?** Because the sum $\sum_k q(z_i = k) \cdot (\dots)$ is a **weighted average** (an expectation under q), and weighted averages interact nicely with concave functions like log via Jensen’s inequality.

Step 3: apply Jensen’s inequality to get a lower bound. Before stating the inequality, let us understand **why we want a lower bound at all**.

Our goal is to find the parameters θ that maximize $\log p(X | \theta)$. But we cannot maximize $\log p(X | \theta)$ directly because it contains a log-of-a-sum (equation (14)) that has no closed-form solution.

Suppose instead we had a simpler function $\mathcal{L}(q, \theta)$ with two properties:

1. **It is a lower bound:** $\mathcal{L}(q, \theta) \leq \log p(X | \theta)$ for all q, θ .
2. **It can be made tight:** there exists a choice of q such that $\mathcal{L}(q, \theta) = \log p(X | \theta)$ exactly.

If we have such a function, we can use the following two-step strategy to maximize $\log p(X | \theta)$:

- **Tighten the bound** (choose q to close the gap): after this step, $\mathcal{L}(q, \theta) = \log p(X | \theta)$ —the bound touches the true objective.
- **Push the bound up** (choose θ to increase $\mathcal{L}$): after this step, $\mathcal{L}$ increases. Since the bound was tight before this step, the true objective $\log p(X | \theta)$ must also increase (or stay the same).

Alternating these two steps gives a monotonically non-decreasing sequence $\log p(X | \theta^{(0)}) \leq \log p(X | \theta^{(1)}) \leq \dots$. **This is the EM algorithm:** the first step is the E-step, the second is the M-step, and $\mathcal{L}$ is the ELBO.

So: we need a lower bound not for its own sake, but because it gives us a **tractable surrogate** that we can optimize in place of the intractable true objective, with the guarantee that each optimization cycle improves (or maintains) the true objective.

Now we construct such a bound using Jensen’s inequality.

Definition 3.1 (Jensen’s inequality). If φ is a **concave** function (curves downward, like $\log$) and $W_1, \dots, W_N$ are non-negative weights summing to 1, then for any values $a_1, \dots, a_N$:

$$\varphi\left(\sum_k W_k a_k\right) \geq \sum_k W_k \varphi(a_k). \quad (16)$$

In words: the function of the weighted average is $\geq$ the weighted average of the function.

We apply (16) to (15) with $\varphi = \log$ (which is concave), $W_k = q(z_i = k)$, and $a_k = p(x_i, z_i = k | \theta) / q(z_i = k)$:

$$\log \sum_k q(z_i = k) \cdot \frac{p(x_i, z_i = k | \theta)}{q(z_i = k)} \geq \sum_k q(z_i = k) \cdot \log \frac{p(x_i, z_i = k | \theta)}{q(z_i = k)}. \quad (17)$$

The log has moved **inside** the sum, at the cost of turning an equality into an inequality ($\geq$). The right-hand side is a lower bound on the left-hand side.

Step 4: split the log of a ratio. Using $\log(A/B) = \log A - \log B$:

$$\begin{aligned} \sum_k q(z_i = k) \log \frac{p(x_i, z_i = k | \theta)}{q(z_i = k)} &= \sum_k q(z_i = k) \log p(x_i, z_i = k | \theta) \\ &\quad - \sum_k q(z_i = k) \log q(z_i = k). \end{aligned} \quad (18)$$

The first sum is the expected joint log-likelihood under q . The second sum (with the minus sign) is the **entropy** of q , written $H[q] = -\sum_k q_k \log q_k$.

Step 5: sum over all data points and define the ELBO. Summing (17)–(18) over $i = 1, \dots, P$, and writing $r_{ik} = q(z_i = k)$:

$$\log p(X | \theta) \geq \underbrace{\sum_{i,k} r_{ik} \log p(x_i, z_i = k | \theta)}_{\text{expected complete-data log-likelihood}} + \underbrace{\left(-\sum_{i,k} r_{ik} \log r_{ik}\right)}_{\text{entropy } H[q]} \equiv \text{ELBO}(q, \theta). \quad (19)$$

Key idea. The ELBO is a **lower bound** on the log marginal likelihood: $\log p(X | \theta) \geq \text{ELBO}(q, \theta)$ for *any* distribution q over the latent variables.

This bound holds because Jensen’s inequality introduced a $\geq$ at Step 3. The bound is tight (equality holds) when $q(z_i = k) = p(z_i = k | x_i, \theta)$ —i.e., when q equals the true posterior. In that case the gap is zero:

$$\log p(X | \theta) - \text{ELBO}(q, \theta) = \text{KL}(q(Z) \| p(Z | X, \theta)) \geq 0, \quad (20)$$

where KL is the Kullback-Leibler divergence (a non-negative measure of “distance” between two distributions; it equals zero if and only if the two distributions are identical).

Intuition 3.2. Why does this help? Because the ELBO has $\log p(x_i, z_i = k | \theta)$ (joint) instead of $\log \sum_k p(x_i, z_i = k | \theta)$ (marginal). The joint factors as $p(z_i = k) \cdot p(x_i | z_i = k)$ (chain rule), and the log of a product is a sum of logs—no more “log of a sum.” This makes the M-step tractable.

3.4 Decomposing the ELBO

To make the ELBO concrete, we need to expand $\log p(x_i, z_i = k | \theta)$. Using the chain rule (10):

$$p(x_i, z_i = k | \theta) = p(z_i = k | \theta) \cdot p(x_i | z_i = k, \theta). \quad (21)$$

Substituting our specific distributions (Categorical prior π_k , Gaussian emission):

$$p(x_i, z_i = k | \theta) = \pi_k \cdot \mathcal{N}(x_i; \mu_k, \Sigma_k). \quad (22)$$

Taking the log: $\log p(x_i, z_i = k | \theta) = \log \pi_k + \log \mathcal{N}(x_i; \mu_k, \Sigma_k)$. Plugging into the ELBO (19):

$$\text{ELBO}(q, \theta) = \sum_{i,k} r_{ik} \log \pi_k + \sum_{i,k} r_{ik} \log \mathcal{N}(x_i; \mu_k, \Sigma_k) - \sum_{i,k} r_{ik} \log r_{ik}, \quad (23)$$

where we wrote $r_{ik} = q(z_i = k)$. The three terms are:

1. **Assignment log-likelihood:** $\sum_{i,k} r_{ik} \log \pi_k$. Prefers configurations where points are assigned to clusters with large π_k .
2. **Data log-likelihood:** $\sum_{i,k} r_{ik} \log \mathcal{N}(x_i; \mu_k, \Sigma_k)$. Prefers configurations where each point is close to its assigned center.
3. **Entropy:** $-\sum_{i,k} r_{ik} \log r_{ik}$. Prefers “soft” (uncertain) assignments. This prevents the trivial solution of putting all mass on one cluster.

4 The EM Algorithm

4.1 E-step: optimize over q

Fix θ . We want $q^* = \arg \max_q \text{ELBO}(q, \theta)$. Since the gap is the KL divergence $\text{KL}(q \| p(Z | X, \theta))$, setting $q^*(z_i = k) = p(z_i = k | x_i, \theta)$ makes $\text{KL} = 0$ and the ELBO equals $\log p(X | \theta)$ exactly.

We compute this using **Bayes’ theorem**.

Definition 4.1 (Bayes’ theorem). For any two events or variables A and B with $p(B) > 0$:

$$p(A | B) = \frac{p(B | A) p(A)}{p(B)}. \quad (24)$$

In words: posterior = (likelihood $\times$ prior) / evidence. The evidence $p(B)$ is a normalizing constant that ensures the posterior sums/integrates to 1.

We apply (24) with $A = (z_i = k)$ and $B = x_i$:

$$p(z_i = k \mid x_i, \theta) = \frac{p(x_i \mid z_i = k, \theta) \cdot p(z_i = k \mid \theta)}{p(x_i \mid \theta)}. \quad (25)$$

Now we substitute each factor using what we know about our model:

- **Prior:** $p(z_i = k \mid \theta) = \pi_k$ (Categorical distribution over components).
- **Likelihood:** $p(x_i \mid z_i = k, \theta) = \mathcal{N}(x_i; \mu_k, \Sigma_k)$ (Gaussian emission of component k).
- **Evidence:** $p(x_i \mid \theta) = \sum_{j=1}^N p(x_i, z_i = j \mid \theta) = \sum_{j=1}^N \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)$ (the marginal density, obtained by summing the joint over all components, as derived in Section 2.1).

Substituting into (25):

$$r_{ik} = p(z_i = k \mid x_i, \theta) = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^N \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}. \quad (26)$$

Intuition 4.1. The responsibility r_{ik} is the posterior probability that point i belongs to cluster k . It is large when x_i is close to μ_k (high $\mathcal{N}$ value) and π_k is large (component is a priori likely). Points equidistant from two centers will have $r_{ik} \approx r_{ij} \approx 0.5$ —soft assignment.

4.2 M-step: optimize over θ

Fix q (i.e., fix the responsibilities r_{ik}). We want $\theta^* = \arg \max_{\theta} \text{ELBO}(q, \theta)$. Since the entropy term $H[q] = -\sum_{i,k} r_{ik} \log r_{ik}$ does not depend on θ (the r_{ik} are fixed numbers), this is equivalent to maximizing the **expected complete-data log-likelihood**:

$$Q(\theta) = \sum_{i,k} r_{ik} \log p(x_i, z_i = k \mid \theta). \quad (27)$$

Now we expand $\log p(x_i, z_i = k \mid \theta)$ using the chain rule:

$$\begin{aligned} \log p(x_i, z_i = k \mid \theta) &= \log [p(z_i = k \mid \theta) \cdot p(x_i \mid z_i = k, \theta)] \\ &= \log p(z_i = k \mid \theta) + \log p(x_i \mid z_i = k, \theta) \quad (\text{log of a product} = \text{sum of logs}) \\ &= \log \pi_k + \log \mathcal{N}(x_i; \mu_k, \Sigma_k). \quad (\text{substituting our specific distributions}) \end{aligned} \quad (28)$$

Plugging back into (27):

$$Q(\theta) = \sum_{i,k} r_{ik} [\log \pi_k + \log \mathcal{N}(x_i; \mu_k, \Sigma_k)]. \quad (29)$$

Update for π_k : Maximize $\sum_k (\sum_i r_{ik}) \log \pi_k$ subject to $\sum_k \pi_k = 1$. By Lagrange multipliers:

$$\pi_k^{\text{new}} = \frac{R_k}{P}, \quad R_k = \sum_{i=1}^P r_{ik}. \quad (30)$$

R_k is the **effective mass** of cluster k —the total soft-assignment weight. The updated π_k is just the fraction of total mass in cluster k .

Update for μ_k : Maximize $\sum_i r_{ik} \log \mathcal{N}(x_i; \mu_k, \Sigma_k)$ over μ_k . The log-Gaussian contains $-\frac{1}{2}(x_i - \mu_k)^\top \Sigma_k^{-1}(x_i - \mu_k)$. Taking the gradient:

$$\nabla_{\mu_k} Q = \sum_i r_{ik} \Sigma_k^{-1}(x_i - \mu_k) = 0 \implies \mu_k^{\text{new}} = \frac{\sum_i r_{ik} x_i}{R_k}. \quad (31)$$

The new center is the **responsibility-weighted mean** of the data.

Update for Σ_k : Maximize over Σ_k . This involves the log-determinant and the quadratic form. Taking the matrix derivative and setting to zero:

$$\Sigma_k^{\text{new}} = \frac{1}{R_k} \sum_i r_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^\top. \quad (32)$$

The new covariance is the **responsibility-weighted scatter** around the new mean.

Key idea. EM monotonicity: Each E-step makes $\text{ELBO}(q, \theta) = \log p(X | \theta)$ (closes the KL gap). Each M-step maximizes $Q(\theta)$ over θ , which increases ELBO (and hence $\log p(X | \theta)$). The sequence of marginal likelihoods is non-decreasing:

$$\log p(X | \theta^{(0)}) \leq \log p(X | \theta^{(1)}) \leq \log p(X | \theta^{(2)}) \leq \dots$$

Convergence to a local maximum is guaranteed.

5 Adding Bayesian Priors

5.1 Why priors?

MLE can overfit: with N clusters and P points, if a cluster contains very few points, its covariance estimate $\hat{\Sigma}_k$ becomes singular (not invertible). Bayesian priors **regularize** by encoding prior beliefs about parameter ranges. The ELBO becomes:

$$\text{ELBO}(q, \theta) = \mathbb{E}_q[\log p(X, Z | \theta)] + H[q] + \log p(\theta), \quad (33)$$

where $\log p(\theta)$ is the log-prior on all parameters. The M-step now maximizes $Q(\theta) + \log p(\theta)$, which is the **MAP (maximum a posteriori)** objective.

5.2 The Dirichlet prior on mixing weights

The mixing weights $\pi = (\pi_1, \dots, \pi_N)$ live on the probability simplex ($\pi_k \geq 0$, $\sum_k \pi_k = 1$). The natural prior is the **Dirichlet distribution**:

$$p(\pi | \alpha_0) = \frac{\Gamma(N\alpha_0)}{\Gamma(\alpha_0)^N} \prod_{k=1}^N \pi_k^{\alpha_0-1}. \quad (34)$$

The parameter $\alpha_0 > 0$ controls the prior's character:

- $\alpha_0 = 1$: uniform prior on the simplex (all weight allocations equally likely).
- $\alpha_0 > 1$: prefers π near uniform ($\pi_k \approx 1/N$ for all k). Discourages empty clusters.
- $\alpha_0 < 1$: **sparse-inducing**. Prefers π concentrated on a few components, with many $\pi_k \approx 0$.

Key idea. We use $\alpha_0 = 0.5 < 1$ to encourage the model to **prune unused clusters**. Starting with N larger than needed, the Dirichlet prior drives $\pi_k \rightarrow 0$ for redundant components. This is automatic model-complexity selection.

MAP update for π : Adding $\log p(\pi) = \sum_k (\alpha_0 - 1) \log \pi_k + \text{const}$ to the Lagrangian:

$$\pi_k^{\text{new}} = \frac{R_k + \alpha_0 - 1}{P + N(\alpha_0 - 1)}. \quad (35)$$

When $\alpha_0 < 1$ and R_k is small, π_k can go negative; we clamp to $\pi_k \geq 10^{-10}$ and renormalize. In practice, clusters with $R_k < 1$ are pruned (removed from the model entirely).

5.3 The Normal-Inverse-Wishart (NIW) prior on centers and covariances

For the center c_k and covariance Σ_k of each cluster, we use the **NIW prior**:

$$\Sigma_k \sim \mathcal{IW}(\Psi_0, \nu_0), \quad (36)$$

$$c_k \mid \Sigma_k \sim \mathcal{N}(\mu_0, \Sigma_k / \kappa_0), \quad (37)$$

where $\Psi_0 \in \mathbb{R}^{d \times d}$ (positive definite scale matrix), $\nu_0 > d - 1$ (degrees of freedom), $\mu_0 \in \mathbb{R}^d$ (prior mean), $\kappa_0 > 0$ (prior precision scaling).

Intuition 5.1. The NIW prior encodes:

- μ_0 : “we expect cluster centers near this point” (set to the data mean).
- κ_0 : “how strongly we believe that” (small κ_0 = weak prior = centers free to move; large κ_0 = strong pull toward μ_0).
- Ψ_0 : “we expect covariances of roughly this scale” (set proportional to identity).
- ν_0 : “how confident we are about the covariance” (small ν_0 = vague prior; large = confident).

Why “conjugate”? A prior is **conjugate** to a likelihood if the posterior has the same functional form as the prior. For a Gaussian likelihood with NIW prior, the posterior on (c_k, Σ_k) is also NIW with updated parameters:

$$\kappa_n = \kappa_0 + R_k, \quad \nu_n = \nu_0 + R_k, \quad (38)$$

$$\mu_n = \frac{\kappa_0 \mu_0 + R_k \bar{x}_k}{\kappa_n}, \quad (39)$$

$$\Psi_n = \Psi_0 + S_k + \frac{\kappa_0 R_k}{\kappa_n} (\bar{x}_k - \mu_0)(\bar{x}_k - \mu_0)^\top, \quad (40)$$

where $\bar{x}_k = \frac{1}{R_k} \sum_i r_{ik} x_i$ is the responsibility-weighted mean and $S_k = \sum_i r_{ik} (x_i - \bar{x}_k)(x_i - \bar{x}_k)^\top$ is the weighted scatter.

Conjugacy is important because it gives **closed-form marginal likelihoods** (Section 9).

MAP updates for c_k and Σ_k : Setting the gradient of $Q(\theta) + \log p(\theta)$ to zero for c_k :

$$(\kappa_0 \Sigma_k^{-1} + R_k \Sigma_k^{-1}) c_k^{\text{new}} = \kappa_0 \Sigma_k^{-1} \mu_0 + \Sigma_k^{-1} \sum_i r_{ik} x_i. \quad (41)$$

Since Σ_k^{-1} factors out (and we often write $\Lambda_0 = \kappa_0 \Sigma_k^{-1}$ or a fixed matrix):

$$c_k^{\text{new}} = \frac{\kappa_0 \mu_0 + \sum_i r_{ik} x_i}{\kappa_0 + R_k} = \frac{\kappa_0 \mu_0 + R_k \bar{x}_k}{\kappa_n}. \quad (42)$$

This is a **weighted average** of the prior mean μ_0 and the data mean $\bar{x}_k$, with weights κ_0 (prior) and R_k (data). More data $\rightarrow$ center follows data; less data $\rightarrow$ center stays near μ_0 .

For Σ_k , the MAP mode of the Inverse-Wishart posterior is:

$$\Sigma_k^{\text{new}} = \frac{\Psi_n}{\nu_n + d + 1} = \frac{\Psi_0 + S_k + \text{conflict}}{\nu_0 + R_k + d + 1}. \quad (43)$$

The “conflict” term $\frac{\kappa_0 R_k}{\kappa_n} (\bar{x}_k - \mu_0)(\bar{x}_k - \mu_0)^\top$ penalizes disagreement between the prior mean and data mean. In our implementation, we use a simplified version where Λ_0 is a fixed precision matrix:

$$\Sigma_k^{\text{new}} = \frac{\Psi_0 + \sum_i r_{ik} (x_i - c_k^{\text{new}})(x_i - c_k^{\text{new}})^\top}{\nu_0 + R_k + d + 1} + \lambda I. \quad (44)$$

The λI term (typically 10^{-6}) prevents numerical singularity.

6 Our Model: Adding the Residual Factor

Key idea. Connection to Cluster-Weighted Models. The model we are about to define is an instance of the **Cluster-Weighted Model** (CWM), a framework introduced by Gershenfeld (1997) and formalized statistically by Ingrassia, Minotti, and Vittadini (2012). A CWM models the joint density $p(x, y) = \sum_k \pi_k p(x|k) p(y|x, k)$ —combining a marginal density on inputs $p(x|k)$ with a conditional regression on outputs $p(y|x, k)$. In the Linear Gaussian CWM, both are Gaussian with an affine conditional mean.

Our model is a CWM where x is a state vector and $y = f(x)$ is the vector field. The standard data-driven affine local model (our Variant B) is exactly a Linear Gaussian CWM. Our contributions are: (i) a physics-constrained variant where the local model is the analytic Taylor expansion (Variant A, no precedent in CWM literature), (ii) a Koopman-theoretic variant with lifted-space generators (Variant C), and (iii) application to dynamical-systems phase-space partitioning, an entirely new domain for CWM.

Sections 1–5 presented standard Bayesian GMM machinery. From this point on, everything is specific to the CWM formulation for dynamical systems.

6.1 The generative model

We augment the standard GMM with a **second observation per data point**: the vector-field value $f(x_i)$. The generative story for each point:

$$z_i \sim \text{Cat}(\pi), \quad (45)$$

$$x_i \mid z_i = k \sim \mathcal{N}(c_k, \Sigma_k), \quad (46)$$

$$f_{\text{obs}}(x_i) \mid x_i, z_i = k \sim \mathcal{N}(L_k(x_i), \sigma^2 I). \quad (47)$$

Line (46) is the proximity model (standard GMM). Line (47) is the residual model (novel): it says that the observed vector field $f_{\text{obs}}(x_i)$ is drawn from a Gaussian centered at the local model’s prediction $L_k(x_i)$, with isotropic noise $\sigma^2 I$.

What is $L_k(x_i)$? This is the local model’s prediction of the vector field at x_i . It depends on:

- The cluster’s center c_k and local-model parameters (e.g., Jacobian $\mathbf{J}_k$, or Koopman operator $\mathbf{M}_k$).
- The input point x_i itself.

Its specific form depends on the variant (Taylor, LS, or EDMD). The framework is agnostic to this choice.

6.2 The joint observation likelihood

For a single data point $(x_i, f_{\text{obs}}(x_i))$ assigned to cluster k , we want the joint density $p(x_i, f_{\text{obs}}(x_i) \mid z_i = k, \theta)$. We build it in three steps.

Step 1: apply the chain rule of probability. For any two variables A and B :

$$p(A, B) = p(A) \cdot p(B \mid A). \quad (48)$$

This decomposes a joint density into a marginal times a conditional. Applying this with $A = x_i$ and $B = f_{\text{obs}}(x_i)$, all conditioned on $z_i = k$:

$$p(x_i, f_{\text{obs}}(x_i) \mid z_i = k, \theta) = \underbrace{p(x_i \mid z_i = k, \theta)}_{\text{factor 1}} \cdot \underbrace{p(f_{\text{obs}}(x_i) \mid x_i, z_i = k, \theta)}_{\text{factor 2}}. \quad (49)$$

Step 2: identify each factor from our generative model.

- **Factor 1** (proximity): From (46), the generative model says $x_i \mid z_i = k \sim \mathcal{N}(c_k, \Sigma_k)$. So:

$$p(x_i \mid z_i = k, \theta) = \mathcal{N}(x_i; c_k, \Sigma_k). \quad (50)$$

- **Factor 2** (residual): From (47), the generative model says $f_{\text{obs}}(x_i) \mid x_i, z_i = k \sim \mathcal{N}(L_k(x_i), \sigma^2 I)$. So:

$$p(f_{\text{obs}}(x_i) \mid x_i, z_i = k, \theta) = \mathcal{N}(f_{\text{obs}}(x_i); L_k(x_i), \sigma^2 I). \quad (51)$$

Step 3: substitute into the chain rule (49).

$$p(x_i, f_{\text{obs}}(x_i) \mid z_i = k, \theta) = \underbrace{\mathcal{N}(x_i; c_k, \Sigma_k)}_{\text{proximity}} \cdot \underbrace{\mathcal{N}(f_{\text{obs}}(x_i); L_k(x_i), \sigma^2 I)}_{\text{residual}}. \quad (52)$$

This is a product of two Gaussian densities over **different random variables**: the first is a density over $x_i \in \mathbb{R}^d$, the second is a density over $f_{\text{obs}}(x_i) \in \mathbb{R}^d$. Together they define a density over the pair $(x_i, f_{\text{obs}}(x_i)) \in \mathbb{R}^{2d}$.

Derivation. Why is (48) valid here? In the generative model, x_i is drawn first (from the proximity model), then $f_{\text{obs}}(x_i)$ is drawn conditionally on x_i (from the residual model). The chain rule of probability is a universal identity—it holds for any joint distribution, not just ours. We are simply choosing to decompose $p(x_i, f_{\text{obs}})$ into $p(x_i) \cdot p(f_{\text{obs}} \mid x_i)$ rather than the other way around, because this matches the order of our generative process.

6.3 The log-likelihood of the joint observation

Taking the log:

$$\begin{aligned} \log p(x_i, f_{\text{obs}}(x_i) \mid z_i = k) &= \underbrace{-\frac{d}{2} \log 2\pi - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x_i - c_k)^\top \Sigma_k^{-1} (x_i - c_k)}_{\text{proximity: depends on } x_i, c_k, \Sigma_k} \\ &\quad + \underbrace{-\frac{d}{2} \log(2\pi\sigma^2) - \frac{\|f_{\text{obs}}(x_i) - L_k(x_i)\|^2}{2\sigma^2}}_{\text{residual: depends on } f_{\text{obs}}(x_i), L_k(x_i), \sigma^2}. \end{aligned} \quad (53)$$

Key idea. The log-likelihood is the **sum of two quadratic penalties**:

1. **Proximity penalty:** $(x_i - c_k)^\top \Sigma_k^{-1} (x_i - c_k)$ — how far is x_i from the center in the Σ_k -metric?
2. **Residual penalty:** $\|f_{\text{obs}}(x_i) - L_k(x_i)\|^2 / \sigma^2$ — how badly does the local model predict f at this point?

The relative importance of these two penalties is controlled by the ratio of their scales: $\lambda(\Sigma_k^{-1})$ vs. $1/\sigma^2$. When σ^2 is small, the residual penalty dominates; when σ^2 is large, the proximity penalty dominates.

6.4 The role of the residual scale

σ^2 is the single most important hyperparameter in the framework. It controls the **balance between the two forces**:

- $\sigma^2 \rightarrow \infty$: The residual penalty $\|\varepsilon_k\|^2 / (2\sigma^2) \rightarrow 0$. The joint likelihood reduces to the proximity factor alone. **You recover standard GMM.** Clusters form purely by geometry, ignoring dynamics.
- $\sigma^2 \rightarrow 0$: The residual penalty dominates everything. A point is assigned to whichever cluster has the smallest $\|\varepsilon_k\|$, regardless of geometric proximity. **You get a piecewise regression** where clusters form by model fit alone.
- $\sigma^2 \sim \text{median}(\|\varepsilon_k\|^2)/d$: Both forces contribute comparably. **This is the interesting regime.** Clusters form by a combination of geometric proximity and model quality.

Auto-calibration: We set σ^2 from the initial GMM partition’s residuals:

$$\sigma^2 = \max\left(\frac{\text{median}_i \|\varepsilon_k(x_i)\|^2}{d}, 10^{-3}\right). \quad (54)$$

This ensures the residual penalty is on the same scale as the actual linearization errors in the initial (geometric) partition. The floor 10^{-3} prevents numerical instability when residuals are very small (e.g., on polynomial systems where the local model is exact).

Could σ^2 be learned? Yes. The M-step update would be:

$$\sigma^{2,\text{new}} = \frac{\sum_{i,k} r_{ik} \|\varepsilon_k(x_i)\|^2}{d \cdot P}. \quad (55)$$

But this can collapse to $\sigma^2 \rightarrow \infty$ (residual becomes irrelevant) or $\sigma^2 \rightarrow 0$ (proximity becomes irrelevant), both of which are local optima of the ELBO. A prior $\sigma^2 \sim \text{InverseGamma}(a_0, b_0)$ would prevent collapse but adds two more hyperparameters. We defer this and use the auto-calibrated fixed value.

7 E-step for Our Model

The E-step computes the posterior $p(z_i = k \mid x_i, f_{\text{obs}}(x_i), \theta)$ —the probability that point i belongs to cluster k , given *both* the position x_i and the observed vector field $f_{\text{obs}}(x_i)$. We derive this in three steps.

Step 1: apply Bayes’ theorem (24). Set $A = (z_i = k)$ and $B = (x_i, f_{\text{obs}}(x_i))$ (i.e., the “data” is now the *pair* of observations):

$$p(z_i = k \mid x_i, f_{\text{obs}}(x_i), \theta) = \frac{p(x_i, f_{\text{obs}}(x_i) \mid z_i = k, \theta) \cdot p(z_i = k \mid \theta)}{p(x_i, f_{\text{obs}}(x_i) \mid \theta)}. \quad (56)$$

Step 2: identify each factor from our model.

- **Likelihood:** $p(x_i, f_{\text{obs}}(x_i) \mid z_i = k, \theta)$ is the joint observation likelihood (52), which we already derived: $= \mathcal{N}(x_i; c_k, \Sigma_k) \cdot \mathcal{N}(f_{\text{obs}}(x_i); L_k(x_i), \sigma^2 I)$.
- **Prior:** $p(z_i = k \mid \theta) = \pi_k$ (Categorical).
- **Evidence:** $p(x_i, f_{\text{obs}}(x_i) \mid \theta) = \sum_{j=1}^N p(x_i, f_{\text{obs}}(x_i), z_i = j \mid \theta)$ (marginalize over all clusters, same pattern as Section 2.1). Using the chain rule on each term: $= \sum_{j=1}^N \pi_j \cdot \mathcal{N}(x_i; c_j, \Sigma_j) \cdot \mathcal{N}(f_{\text{obs}}(x_i); L_j(x_i), \sigma^2 I)$.

Step 3: substitute into (56). Writing $\varepsilon_k(x_i) = f_{\text{obs}}(x_i) - L_k(x_i)$:

$$r_{ik} = p(z_i = k \mid x_i, f_{\text{obs}}(x_i), \theta) = \frac{\pi_k \mathcal{N}(x_i; c_k, \Sigma_k) \mathcal{N}(\varepsilon_k(x_i); 0, \sigma^2 I)}{\sum_{j=1}^N \pi_j \mathcal{N}(x_i; c_j, \Sigma_j) \mathcal{N}(\varepsilon_j(x_i); 0, \sigma^2 I)}. \quad (57)$$

Compare to standard GMM (26): the only difference is the extra factor $\mathcal{N}(\varepsilon_k(x_i); 0, \sigma^2 I)$ in both numerator and denominator. This factor **upweights** clusters where the residual is small (good local fit) and **downweights** clusters where the residual is large (bad local fit).

Numerical implementation: We work in log-space to avoid underflow:

$$\log \tilde{r}_{ik} = \log \pi_k + \log \mathcal{N}(x_i; c_k, \Sigma_k) + \log \mathcal{N}(\varepsilon_k(x_i); 0, \sigma^2 I), \quad (58)$$

$$\log r_{ik} = \log \tilde{r}_{ik} - \text{logsumexp}_k \log \tilde{r}_{ik}, \quad (59)$$

where $\text{logsumexp}_k(a_k) = \log \sum_k \exp(a_k)$, computed via the max-shift trick for numerical stability.

8 M-step for Our Model

The M-step maximizes the expected complete-data log-likelihood $Q(\theta)$ plus log-priors over θ . We build Q step by step.

The general form of Q is (27): $Q(\theta) = \sum_{i,k} r_{ik} \log p(x_i, f_{\text{obs}}(x_i), z_i = k \mid \theta)$. Using the chain rule to decompose the joint:

$$\begin{aligned} \log p(x_i, f_{\text{obs}}(x_i), z_i = k \mid \theta) &= \log p(z_i = k \mid \theta) + \log p(x_i \mid z_i = k, \theta) + \log p(f_{\text{obs}}(x_i) \mid x_i, z_i = k, \theta) \\ &= \log \pi_k + \log \mathcal{N}(x_i; c_k, \Sigma_k) + \log \mathcal{N}(\varepsilon_k(x_i); 0, \sigma^2 I). \end{aligned} \quad (60)$$

The first two terms are identical to standard GMM. The third term—the log-residual—is our novel contribution. Substituting into Q :

$$Q(\theta) = \underbrace{\sum_{i,k} r_{ik} \log \pi_k}_{Q_\pi} + \underbrace{\sum_{i,k} r_{ik} \log \mathcal{N}(x_i; c_k, \Sigma_k)}_{Q_{\text{prox}}} + \underbrace{\sum_{i,k} r_{ik} \log \mathcal{N}(\varepsilon_k(x_i); 0, \sigma^2 I)}_{Q_{\text{res}}}. \quad (61)$$

The Q_π and Q_{prox} terms are identical to standard GMM. The novel Q_{res} term is:

$$Q_{\text{res}} = -\frac{Pd}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i,k} r_{ik} \|\varepsilon_k(x_i)\|^2. \quad (62)$$

The π and Σ_k updates are the same as in Sections 4–5. What changes is the **center update**: Q_{res} depends on c_k through $\varepsilon_k(x_i) = f(x_i) - L_k(x_i)$, and L_k depends on c_k . How this dependence is handled defines the three variants.

8.1 Variant A: Taylor-analytic

Here $L_k(x) = f(c_k) + \mathbf{J}(c_k)(x - c_k)$, where f and $\mathbf{J}$ are evaluated analytically. The center c_k enters Q_{res} through both $f(c_k)$ and $\mathbf{J}(c_k)$. In the M-step, we:

1. Update c_k using only the proximity gradient:

$$(\Lambda_0 + R_k \Sigma_k^{-1}) c_k^{\text{new}} = \Lambda_0 \mu_0 + \Sigma_k^{-1} \sum_i r_{ik} x_i. \quad (63)$$

This is the standard GMM center update with NIW prior.

2. **Re-tie**: set $f(c_k) \leftarrow f(c_k^{\text{new}})$, $\mathbf{J}(c_k) \leftarrow \mathbf{J}(c_k^{\text{new}})$ analytically.

Remark 8.1 (Why this is approximate). The gradient $\nabla_{c_k} Q_{\text{res}}$ is nonzero—it contains a term from $\partial \varepsilon_k / \partial c_k$. By ignoring it in step 1, we are not fully maximizing Q over c_k . This makes the update a **generalized EM (GEM)** step: it increases Q but does not maximize it. The ELBO is still non-decreasing at each step (GEM guarantees this), but it may not converge to the same point as full maximization.

Additionally, step 2 “re-ties” $f(c_k)$ and $\mathbf{J}(c_k)$ to the analytic values at the new center. This changes the residual in a way that is *not* accounted for by the gradient—it’s a separate heuristic update. In practice, at small N (large clusters, large σ^2), this is harmless. At large N (small clusters, small σ^2), the residual term dominates and the non-variational re-tying can decrease the ELBO.

8.2 Variant B: Taylor-LS

Here $L_k(x) = f_k + \mathbf{J}_k(x - c_k)$, where $(f_k, \mathbf{J}_k)$ are **free parameters** fit by weighted least squares. We now have three parameter groups to update: $(f_k, \mathbf{J}_k)$, c_k , and Σ_k . We update them sequentially (coordinate ascent):

Step 1: update $(f_k, \mathbf{J}_k)$ by weighted LS. The residual is $\varepsilon_k(x_i) = f_{\text{obs}}(x_i) - f_k - \mathbf{J}_k(x_i - c_k)$. Fix c_k . The Q_{res} term is:

$$Q_{\text{res}}(f_k, \mathbf{J}_k) = -\frac{1}{2\sigma^2} \sum_i r_{ik} \|f_{\text{obs}}(x_i) - f_k - \mathbf{J}_k(x_i - c_k)\|^2 + \text{const}. \quad (64)$$

This is a **weighted least-squares** problem. Define:

$$\phi_i = \begin{bmatrix} 1 \\ x_i - c_k \end{bmatrix} \in \mathbb{R}^{d+1}, \quad Z = \begin{bmatrix} \phi_1^\top \\ \vdots \\ \phi_P^\top \end{bmatrix} \in \mathbb{R}^{P \times (d+1)}, \quad (65)$$

$$W = \text{diag}(r_{1k}, \dots, r_{Pk}), \quad Y = \begin{bmatrix} f_{\text{obs}}(x_1)^\top \\ \vdots \\ f_{\text{obs}}(x_P)^\top \end{bmatrix} \in \mathbb{R}^{P \times d}. \quad (66)$$

The optimal $B = [f_k^\top; \mathbf{J}_k^\top] \in \mathbb{R}^{(d+1) \times d}$ satisfies the normal equations:

$$(Z^\top W Z + \lambda I) B = Z^\top W Y. \quad (67)$$

The ridge λI (typically $\lambda = 10^{-6}$) regularizes against ill-conditioning when R_k is small.

Derivation. Take $\nabla_{f_k} Q_{\text{res}} = 0$:

$$\sum_i r_{ik} (f_{\text{obs}}(x_i) - f_k - \mathbf{J}_k(x_i - c_k)) = 0 \implies R_k f_k = \sum_i r_{ik} f_{\text{obs}}(x_i) - \mathbf{J}_k \sum_i r_{ik} (x_i - c_k).$$

This, together with the analogous equation for $\mathbf{J}_k$, is equivalent to the normal equations (67).

Step 2: update c_k (full gradient, including residual). Now fix the new $(f_k, \mathbf{J}_k)$ and optimize c_k . The Q -function depends on c_k through both the proximity term and the residual term. We need:

$$\nabla_{c_k} Q = \nabla_{c_k} Q_{\text{prox}} + \nabla_{c_k} Q_{\text{res}} + \nabla_{c_k} \log p(c_k) = 0. \quad (68)$$

Derivation. Proximity gradient: Q_{prox} contains $-\frac{1}{2} \sum_i r_{ik} (x_i - c_k)^\top \Sigma_k^{-1} (x_i - c_k)$. Taking the gradient:

$$\nabla_{c_k} Q_{\text{prox}} = \Sigma_k^{-1} \sum_i r_{ik} (x_i - c_k) = \Sigma_k^{-1} (S_x - R_k c_k),$$

where $S_x = \sum_i r_{ik} x_i$.

Residual gradient: Q_{res} contains $-\frac{1}{2\sigma^2} \sum_i r_{ik} \|\varepsilon_k(x_i)\|^2$ with $\varepsilon_k(x_i) = f(x_i) - f_k - \mathbf{J}_k(x_i - c_k)$. Note $\frac{\partial \varepsilon_k}{\partial c_k} = \mathbf{J}_k$ (since $\frac{\partial}{\partial c_k} [-\mathbf{J}_k(x_i - c_k)] = \mathbf{J}_k$). So:

$$\begin{aligned} \nabla_{c_k} Q_{\text{res}} &= -\frac{1}{\sigma^2} \sum_i r_{ik} \mathbf{J}_k^\top \varepsilon_k(x_i) \\ &= -\frac{1}{\sigma^2} \mathbf{J}_k^\top [S_f - R_k f_k - \mathbf{J}_k (S_x - R_k c_k)], \end{aligned} \quad (69)$$

where $S_f = \sum_i r_{ik} f_{\text{obs}}(x_i)$.

Prior gradient: $\log p(c_k) \supset -\frac{1}{2} (c_k - \mu_0)^\top \Lambda_0 (c_k - \mu_0)$, so $\nabla_{c_k} \log p(c_k) = \Lambda_0 (\mu_0 - c_k)$.

Setting the total gradient to zero:

$$\Lambda_0 (\mu_0 - c_k) + \Sigma_k^{-1} (S_x - R_k c_k) - \frac{1}{\sigma^2} \mathbf{J}_k^\top [S_f - R_k f_k - \mathbf{J}_k (S_x - R_k c_k)] = 0. \quad (70)$$

Collecting all terms proportional to c_k :

$$\underbrace{\left[\Lambda_0 + R_k \Sigma_k^{-1} + \frac{R_k}{\sigma^2} \mathbf{J}_k^\top \mathbf{J}_k \right]}_{\text{LHS}} c_k^{\text{new}} = \underbrace{\Lambda_0 \mu_0 + \Sigma_k^{-1} S_x - \frac{1}{\sigma^2} \mathbf{J}_k^\top (S_f - R_k f_k - \mathbf{J}_k S_x)}_{\text{RHS}}. \quad (71)$$

Key idea. Equation (71) has three contributions to the left-hand side (the “effective precision” of the center):

1. Λ_0 : pull toward the prior mean μ_0 .
2. $R_k \Sigma_k^{-1}$: pull toward the data mean S_x/R_k (standard GMM term).
3. $(R_k/\sigma^2) \mathbf{J}_k^\top \mathbf{J}_k$: pull toward the point that minimizes the residual (novel term).

Variant A (Taylor-analytic) drops the third term entirely. This is what makes it approximate.

Step 3: update Σ_k . Same as standard GMM with NIW posterior (no residual dependence on Σ_k):

$$\Sigma_k^{\text{new}} = \frac{\Psi_0 + \sum_i r_{ik} (x_i - c_k^{\text{new}})(x_i - c_k^{\text{new}})^\top}{\nu_0 + R_k + d + 1} + \lambda I. \quad (72)$$

8.3 Variant C: Local EDMD

Here $L_k(x) = [\mathbf{M}_k \Phi(x - c_k)]_{1:d}$, where $\Phi: \mathbb{R}^d \rightarrow \mathbb{R}^M$ is a polynomial lifting and $\mathbf{M}_k \in \mathbb{R}^{M \times M}$ is the local Koopman generator.

What is the polynomial lifting? For degree p in d dimensions, $\Phi(u)$ is the vector of all monomials $u_1^{a_1} \cdots u_d^{a_d}$ with $\sum_j a_j \leq p$. For example, with $d = 2, p = 2$:

$$\Phi(u_1, u_2) = (1, u_1, u_2, u_1^2, u_1 u_2, u_2^2)^\top \in \mathbb{R}^6. \quad (73)$$

The total number of monomials is $M = \binom{d+p}{p}$.

Why $[\mathbf{M}_k \Phi]_{1:d}$ gives $\hat{f}$: The first entry of Φ is the constant 1; entries 2 through $d+1$ are the linear monomials $u_1, \dots, u_d$. Their time derivatives are $\dot{u}_j = \dot{x}_j = f_j(x)$. The Koopman generator $\mathbf{M}_k$ is fit so that $\mathbf{M}_k \Phi(u) \approx \dot{\Phi}(u)$, where $\dot{\Phi}$ is the vector of time derivatives of all monomials. Entries 2 through $d+1$ of $\dot{\Phi}$ are exactly $f_1(x), \dots, f_d(x)$. So:

$$[\mathbf{M}_k \Phi(x - c_k)]_j \approx \dot{\Phi}_j = f_j(x), \quad j = 1, \dots, d. \quad (74)$$

M-step for $\mathbf{M}_k$: The generator is fit by weighted least squares. The target for each data point is $\dot{\Phi}(x_i - c_k) = \nabla \Phi(x_i - c_k) \cdot f(x_i)$, the Lie derivative of Φ along f . The feature is $\Phi(x_i - c_k)$. With weights r_{ik} :

$$G_k = \sum_i r_{ik} \Phi(x_i - c_k) \Phi(x_i - c_k)^\top \in \mathbb{R}^{M \times M}, \quad (75)$$

$$A_k = \sum_i r_{ik} \dot{\Phi}(x_i - c_k) \Phi(x_i - c_k)^\top \in \mathbb{R}^{M \times M}, \quad (76)$$

$$\mathbf{M}_k = A_k (G_k + \lambda I)^{-1}. \quad (77)$$

Derivation. The objective is $\min_{\mathbf{M}_k} \sum_i r_{ik} \|\dot{\Phi}_i - \mathbf{M}_k \Phi_i\|^2$. This is a weighted linear regression in $\mathbb{R}^M$: the matrix $\mathbf{M}_k$ is the “weight matrix.” Taking the matrix derivative:

$$\nabla_{\mathbf{M}_k} = -2 \sum_i r_{ik} (\dot{\Phi}_i - \mathbf{M}_k \Phi_i) \Phi_i^\top = 0 \implies \mathbf{M}_k \sum_i r_{ik} \Phi_i \Phi_i^\top = \sum_i r_{ik} \dot{\Phi}_i \Phi_i^\top.$$

This gives $\mathbf{M}_k G_k = A_k$, hence $\mathbf{M}_k = A_k G_k^{-1}$.

c_k **update:** In this variant we use the proximity-only update (63). The residual gradient w.r.t. c_k involves $\nabla_{c_k} \Phi(x_i - c_k)$, which is a tensor of polynomial derivatives. Deriving the closed-form is possible but algebraically heavy and we defer it. This makes the local-EDMD M-step a GEM step for c_k , analogous to Variant A.

9 Model Selection

9.1 How many clusters?

Given the model with N clusters, we want to compare models with different N . The Bayesian approach computes the **marginal likelihood** $p(X, F \mid N)$ —the probability of the data under model N after integrating out all parameters.

9.2 The NIW marginal likelihood (derivation)

For a single cluster k with points $\{x_i : z_i = k\}$, the marginal likelihood integrates out (c_k, Σ_k) :

$$p(X_k) = \int p(X_k \mid c_k, \Sigma_k) p(c_k, \Sigma_k) dc_k d\Sigma_k. \quad (78)$$

Because the NIW prior is conjugate to the Gaussian likelihood, this integral has a closed form:

$$\begin{aligned} \log p(X_k) &= \frac{d}{2} \log \frac{\kappa_0}{\kappa_n} + \frac{\nu_0}{2} \log |\Psi_0| - \frac{\nu_n}{2} \log |\Psi_n| \\ &\quad + \log \Gamma_d\left(\frac{\nu_n}{2}\right) - \log \Gamma_d\left(\frac{\nu_0}{2}\right) - \frac{R_k d}{2} \log \pi, \end{aligned} \quad (79)$$

where Γ_d is the multivariate gamma function $\Gamma_d(a) = \pi^{d(d-1)/4} \prod_{j=1}^d \Gamma(a + (1-j)/2)$.

In our model, we add the residual penalty term:

$$\log p(X_k, F_k) = \log p(X_k) - \frac{1}{2\sigma^2} \sum_{i: z_i = k} \|\varepsilon_k(x_i)\|^2. \quad (80)$$

The total log marginal is:

$$\log p(X, F \mid N) = \underbrace{\log \frac{\Gamma(N\alpha_0)}{\Gamma(P + N\alpha_0)} \prod_k \frac{\Gamma(R_k + \alpha_0)}{\Gamma(\alpha_0)}}_{\text{Dirichlet-Categorical marginal}} + \sum_k \log p(X_k, F_k). \quad (81)$$

The model with the highest $\log p(X, F \mid N)$ is preferred.

10 Summary: What Each Piece Does

Object	What it controls	How it's determined
π_k	Prior probability of cluster k	MAP with Dirichlet prior (Eq. 35)
c_k	Center of cluster k	MAP with NIW prior + residual gradient (Eq. 71)
Σ_k	Shape/size of cluster k	NIW posterior mode (scatter around c_k)
σ^2	Balance: proximity vs. residual	Auto-calibrated from initial residuals; fixed during EM
L_k (local model)	Vector-field prediction in cluster k	Depends on variant (analytic $\mathbf{J}$, LS fit, or EDMD)
r_{ik}	Soft assignment of point i to cluster k	E-step posterior (Eq. 57)
N	Number of clusters	Set high initially; pruned by Dirichlet prior; selected by marginal likelihood

Table 1: Summary of all quantities in the framework.

Hyperparameter	Meaning	Our choice
α_0	Dirichlet concentration	0.5 (sparse-inducing)
μ_0	Prior mean for centers	Data mean $\bar{x}$
Λ_0 (or κ_0)	Prior precision for centers	$0.01I$ (weak: centers free to move)
Ψ_0	Prior scale for covariances	$10I$ or $1I$ (moderate)
ν_0	Prior df for covariances	$d + 2$ (minimal informative)
σ^2	Residual noise scale	Auto-calibrated from median residual
λ (ridge)	Regularization for LS/EDMD	10^{-6} (Taylor-LS) or 10^{-4} (EDMD)

Table 2: Hyperparameters and default values.